SPYWOLF

Security Audit Report

Audit prepared for

PerShare

Completed on

June 23, 2026

TABLE OF CONTENTS

PROJECT INFORMATION

- **Project Name:** PerShare (PerShare V1)
  Token / Short Name: SHARE
  Category: On-chain collective payment pool — a group-buy escrow that lets 2 to 50 members pool stablecoins toward a shared target, validate collectively, and redistribute presale tokens pro-rata.
- **Network:** BNB Smart Chain (BSC Mainnet, chainId 56). Testing and verification were configured for BSC Testnet (chainId 97) ahead of mainnet deployment.
- **Deployed Contracts:** Live on BSC Mainnet across three instances, all verified on BscScan:

  0x646D044703a80E424E20fe78d10ED624383Fa303
  0x35B4BAf4Af02151A76e4e00eA3411EDe495f463a
  0xCA962143652fC02476501f5d32901E5aaaC9F1aD

  These addresses were provided by the client following the audit; the audit itself was performed against the submitted source (PerShare_SmartContracts_For_Audit.zip), confirmed to match the verified on-chain source via BscScan.

- **Token Standard:** BEP-20 / ERC-20.
  Language: Solidity ^0.8.20.
  Compiler: solc 0.8.20, viaIR enabled, optimizer enabled at 200 runs.
  Framework: Hardhat.
  Libraries: OpenZeppelin Contracts v5.6.1 — ReentrancyGuard, Pausable, and Ownable.
- **Files in Scope:** contracts/PerShareV1.sol (the PerShare contract), totalling 541 lines of code.
  Out of Scope: MockUSDT.sol, MockToken.sol, and MockPresaleToken.sol, which are test-only mocks explicitly flagged as not for mainnet deployment.
- **Core Functionality:** Phase 1 handles stablecoin collection, collective member validation, and transfer of funds to an external destination net of a configurable fee. Phase 2 handles reception of BEP-20 presale tokens, member validation, and pro-rata redistribution to contributors in proportion to their contributions.
- **Codebase:** Delivered as PerShare_SmartContracts_For_Audit.zip, containing the contract sources, Hardhat configuration, and package manifest.
  Audit Type: Full manual security review supported by proof-of-concept exploit development on a local EVM.

AUDIT METHODOLOGY(1)

Project Familiarization

- Reviewed the contract's inline specification and the documented Phase 1 / Phase 2 model, including features explicitly deferred to V2 (cancel, pause-driven distribution, internal stablecoin distribution, vesting).
- Mapped the intended group-buy escrow flow (collect → collectively validate → send → receive presale tokens → redistribute) to the implemented code, noting where on-chain accounting and actual token balances diverge.

Automated Analysis

- Static analysis (Slither, Mythril) to surface reentrancy, integer overflow/underflow, uninitialized storage, gas inefficiencies, and unchecked external-call patterns.
- Compilation under the project's own settings (solc 0.8.20, viaIR enabled, optimizer at 200 runs) to confirm the audited bytecode matches the deployment configuration.

Manual Code Review

- Line-by-line review of PerShareV1.sol.
- Verified member management and de-duplication, contribution accounting, validation/threshold logic, fund dispatch and fee calculation, expected-token handling, pro-rata token redistribution, and refund handling.
- Checked checks-effects-interactions ordering, nonReentrant coverage, rounding/dust in distribution math, fee-discount logic, and — critically — whether each payout path withdraws only funds that belong to the share being processed.

Scenario Simulation

- Compiled the contracts and exercised them against an in-memory EVM with proof-of-concept scripts driving real fund flows:
- Standard collection, collective validation, and dispatch to destination net of fee.
- Pro-rata Phase 2 distribution and post-deadline refunds.
- Multiple shares sharing the same stablecoin/token to test cross-share isolation.
- Malicious permissionless share creation pointing expectedToken at funds held for other shares.
- Blacklisted or reverting recipients inside the refund and distribution loops.
- Fee-on-transfer / non-standard token behaviour, and the post-deadline validate-vs-refund race.

AUDIT METHODOLOGY(2)

Risk Assessment

- Classified findings into Critical, High, Medium, Low, and Informational based on exploitability and protocol impact.
- Special focus on fund-custody and solvency risks — cross-share balance draining, payout-loop denial of service, and accounting mismatches that allow one share to be paid from another's deposits.

Tools Used

- Static analyzers: Slither, Mythril.
- Compilation: solc 0.8.20 (viaIR, optimizer 200 runs) via Hardhat.
- Scenario simulation: in-memory EVM (ganache) with ethers.js-driven PoC exploit scripts run against the compiled bytecode.
- Manual reasoning: Solidity 0.8.x checked-arithmetic guarantees and OpenZeppelin ReentrancyGuard, Pausable, and Ownable patterns.

Severity Classification

Severity	Definition
Critical	Direct loss of user funds or protocol-level fund control is achievable by an external actor under realistic deployment conditions, with low cost and high reliability.
High	Significant loss of funds is possible but requires specific preconditions, or causes systemic protocol malfunction without immediate fund loss.
Medium	Limited loss of value, exploitable griefing, or economic logic deviations that meaningfully affect protocol behavior without direct fund theft.
Low	Minor deviations from best practice, dead code, gas inefficiencies, or edge cases with negligible practical impact.
Informational	Code quality observations, documentation gaps, or stylistic notes with no security or economic implication.

FINDINGS

🟥 Critical Risk ✅ RESOLVED

~~Cross-share distribution drains the entire contract balance~~

validateDistribution reads IERC20(tokenAddress).balanceOf(address(this)) — the contract's **entire** balance of a token — and _distributeTokens pays **all of it** to one share's members. There is no per-share token accounting, and setExpectedToken accepts **any** address while createShare is **permissionless**. An attacker can create a throwaway share, point its expectedToken at the stablecoin (or any token the contract is custodying for others), and sweep it.

Impact:
Any user can **drain every other share's funds** under normal operating conditions. PoC result: an honest share holding **1,000 USDT** is emptied by an attacker who started with 100 USDT and ended with **1,099**; the victims' refund() then reverts. This breaks solvency completely.

Code (excerpt):

```
// setExpectedToken — accepts ANY token, incl. the stablecoin
s.expectedToken = tokenAddress;              // L455

// validateDistribution — reads the GLOBAL balance, not this share's
uint256 tokenBalance =
    IERC20(tokenAddress).balanceOf(address(this));   // L345
require(tokenBalance > 0, "PerShare: no tokens received");
...
_distributeTokens(id, tokenAddress, tokenBalance);   // L354

// _distributeTokens — splits that whole balance among one share
uint256 memberTokens = (totalTokens * contribution) / totalContrib; // L383
```

Implemented Fixes:
The team replaced global balance reading with a hermetic, per-share accounting model. Tokens are now credited to a share only through a new depositTokens function that performs an accounted safeTransferFrom and increments an isolated totalTokensReceived counter; validateDistribution and claimDistribution rely exclusively on this counter and never call balanceOf(address(this)). In addition, setExpectedToken now permanently reserves each token to a single share via an isExpectedToken registry, making it impossible for two shares to ever target the same token. SpyWolf re-tested the original exploit against the revised contract and confirmed it reverts with no funds moved — the issue is fully closed.

FINDINGS

🟥 High Risk ✅ RESOLVED

~~Reverting recipient locks refunds and distributions for the whole pool~~

refund() and _distributeTokens() both loop over every member and require() each individual transfer to succeed. If a **single** transfer reverts, the entire transaction reverts and **no one** is paid. USDT and USDC on BSC can **blacklist** addresses, and members may be contracts that revert on receipt — so this is externally triggerable, and also a deliberate griefing vector (a member makes themselves un-payable to freeze everyone else's money).

Impact:
One bad recipient **permanently locks the entire share's funds**. PoC result: two members each deposit 400 (800 in contract); after the deadline one member is blacklisted by the token issuer; the clean member's refund() **reverts** and **all 800 stays locked**. Affects both Phase-1 refunds and Phase-2 token distribution.

Code (excerpt):

```
// refund() — one failing transfer reverts the whole loop
for (uint256 i = 0; i < s.members.length; i++) {
    uint256 amount = contributions[id][member];
    if (amount > 0) {
        contributions[id][member] = 0;
        require(
            IERC20(s.stablecoin).transfer(member, amount),   // L426
            "PerShare: refund failed"
        );
    }
}
// _distributeTokens() has the identical push-payment pattern  // L386
```

Implemented Fixes:
Both payout paths were converted from push loops to a pull pattern. Refunds now use markRefunded (which simply flips the share into refund state) followed by each member independently calling claimRefund, and Phase-2 payouts use claimDistribution per member; each function tracks a per-member claimed flag/amount and transfers only to msg.sender. Because no function iterates transfers across all members, a single blacklisted or reverting recipient can no longer block anyone else. SpyWolf verified that a clean member recovers their full balance even when another member is blacklisted.

FINDINGS

🟨 Medium Risk ✅ RESOLVED

~~Fee-on-transfer / non-standard tokens over-credit contributions~~

contribute() credits contributions[id][msg.sender] and s.totalReceived using the **requested** amount, not the amount the contract actually received. For a fee-on-transfer or rebasing token, the contract records more than it holds. Because all shares share one commingled balance, the inflated share's later _sendFunds/refund can pull the shortfall from **other shares using the same token**. stablecoin is chosen per share and createShare is permissionless, so the token is not guaranteed to be a clean USDT/USDC.

Impact:
A share funded with a fee-on-transfer token records phantom deposits and pays them out from other shares' real balances, **draining honest pools** denominated in the same token and breaking solvency. Standard BSC USDT/USDC are not fee-on-transfer today, which is why this is Medium rather than High.

Code (excerpt):

```
require(
    IERC20(s.stablecoin).transferFrom(msg.sender, address(this), amount), // L260
    "PerShare: transfer failed"
);
contributions[id][msg.sender] += amount;   // credits requested, not received
s.totalReceived               += amount;    // L265-266
```

Implemented Fixes:
Both contribute and depositTokens now measure the actual amount received by taking the contract's token balance before and after the transfer and crediting only the realized delta, rather than the requested amount. All token movements were also migrated to OpenZeppelin's SafeERC20. Combined with the per-share isolation above, a fee-on-transfer or non-standard token can no longer create phantom balances or let one share draw on another's deposits.

FINDINGS

🟨 Medium Risk ✅ RESOLVED

~~validate() lacks a deadline check: post-deadline send/refund race~~

validate() never checks the deadline, while refund() requires the deadline to have passed. Once the deadline elapses **with the target met**, both paths are simultaneously open: a member can validate() to push the funds to the destination, or a member can refund() to pull the whole pool back to contributors. The outcome depends purely on who transacts first.

Impact:
The intended settlement becomes a **front-running race**. A single member can override the group's outcome — forcing a refund when quorum was about to send (stranding a legitimate presale purchase), or completing the send while others expect a refund. No direct theft, but the agreed fund flow is no longer deterministic, and the destination/contributors can be deprived of the result they reasonably expected.

Code (excerpt):

```
// validate() — no deadline guard
function validate(uint256 id) external onlyMember(id) notClosed(id) ... {
    require(s.totalReceived >= s.targetAmount, "PerShare: target not reached"); // L280
    ...                                          // proceeds even after deadline
}

// refund() — only requires the deadline to have passed
require(block.timestamp > s.deadline, "PerShare: deadline not passed yet");     // L413
```

Implemented Fixes:
validate now requires block.timestamp <= deadline, while markRefunded requires block.timestamp > deadline. The two settlement paths are therefore mutually exclusive in time: a send can only occur during the active window and a refund only after it closes, eliminating the front-running race between the two outcomes.

FINDINGS

🟨 Low Risk ✅ RESOLVED

~~No recovery path for stuck or wrong Phase-2 tokens~~

Once _distributeTokens runs, tokensDistributed is set to true and distribution can never run again. If the presale never delivers, delivers the **wrong** token, or sends a **second tranche** after distribution, those tokens are stranded — there is no rescue or re-distribution function. Tokens accidentally sent to the contract address are likewise unrecoverable.

Impact:
Member funds can be **permanently locked** with no path to recover them. This is purely a lock (no theft), and it requires an external mistake or late delivery rather than an attacker, which is why it is rated Low — but for members the loss is total.

Code (excerpt):

```
function _distributeTokens(uint256 id, address tokenAddress, uint256 totalTokens) internal {
    Share storage s = shares[id];
    s.tokensDistributed = true;          // L369 - one-shot, no re-run, no rescue
    ...
}
// no function exists to recover a wrong token or a later tranche
```

Implemented Fixes:
Late or additional presale tranches are now supported by calling depositTokens again (even after distribution), with claimDistribution computing owed amounts cumulatively so members can claim the incremental share. An owner-only sweepLostTokens function recovers tokens sent to the contract by mistake, explicitly excluding any reserved expected-token or registered stablecoin so it can never touch user funds, and a creator-only sweepDust reclaims rounding residue after distribution.

FINDINGS

Informational

Distribution dust assigned to the first contributor

In _distributeTokens, pro-rata division leaves a small remainder ("dust") when the token amount doesn't divide evenly across members. This leftover is always sent to the first contributor in member-array order rather than being split fairly. The amount is negligible (a few token units) and poses no security or solvency risk — it's purely a minor fairness/predictability quirk.

Non-contributing members can reach quorum

Both validate() and validateDistribution() only check isMember — a member who contributed nothing can still cast a validation that counts toward the threshold. With a low threshold relative to the member count, quorum (and therefore the send or distribution) can be triggered entirely by members who never deposited. This is consistent with the M-of-N design, but worth flagging since voting weight is decoupled from funds at stake.

Duplicate member-getter functions

getShareMembers (L505) and getMembers (L538) are functionally identical — both simply return shares[id].members. The redundant second getter adds deployment cost and bytecode size with no benefit; one should be removed to keep the external interface clean.

Redundant zero-initialization of struct fields

In createShare, several Share fields are explicitly assigned their default values — s.totalReceived = 0, s.sent = false, s.refunded = false, s.tokensDistributed = false, s.expectedToken = address(0) (L234–238). Freshly allocated storage is already zero, so these writes only waste gas at creation time. They can be dropped without changing behavior.

CONCLUSION

SpyWolf performed a full security review of the PerShareV1.sol contract, combining line-by-line manual analysis with proof-of-concept exploit development against the compiled bytecode on a local EVM. The review focused on the contract's custody of member funds across its two-phase collect-validate-send-redistribute flow. Following delivery of the initial report, the development team remediated every finding and submitted revised code, which SpyWolf re-reviewed and re-tested.

The initial review identified a Critical vulnerability that allowed any user to drain the entire balance the contract held for every other share, confirmed by a working proof of concept in which an attacker converted 100 USDT into 1,099 by sweeping an honest 1,000 USDT pool, alongside a High-severity push-payment flaw by which a single blacklisted or reverting recipient could permanently lock refunds and distributions for an entire share. Both stemmed from a single root cause: funds were commingled in one balance with no per-share token accounting, and payouts were pushed in loops rather than pulled.

In the revised code the team rearchitected Phase 2 around explicit, per-share token accounting: tokens are credited to a share only through an accounted depositTokens call, distribution relies on an isolated totalTokensReceived counter, the global balanceOf(address(this)) read was removed entirely, and each presale token is permanently bound to a single share. Refunds and distributions were converted to a pull pattern with per-member claim tracking, contribution accounting now measures the realized balance delta to neutralize fee-on-transfer tokens, validate and the refund path were made mutually exclusive in time to close the post-deadline race, and a bounded recovery path was added for stuck or misdirected tokens. SpyWolf re-ran the original Critical and High exploits against the corrected contract and confirmed both now revert with no funds moved; the clean member recovers their full balance even when another member is blacklisted. The two Medium findings, the Low finding, and all informational items were likewise addressed.

Recommendation: All findings from this engagement have been resolved and independently re-verified. The contract follows correct checks-effects-interactions ordering, applies nonReentrant consistently across state-changing entry points, hard-caps the protocol fee, and now isolates per-share custody so that no share can ever draw on another's funds. On the basis of the reviewed and corrected code, SpyWolf considers the contract suitable for mainnet deployment. The development team was responsive and thorough throughout the remediation process.

This report reflects the state of the reviewed code at the time of audit and does not constitute financial advice or a guarantee against future vulnerabilities, particularly in code modified after this review.

SPYWOLF

CRYPTO SECURITY

ABOUT US

We are a growing crypto security agency offering audits, KYCs and consulting services for some of the top names in the crypto industry.

- **OVER 700 SUCCESSFUL CLIENTS**
- **MORE THAN 1000 SCAMS EXPOSED**
- **MILLIONS SAVED IN POTENTIAL FRAUD**
- **PARTNERSHIPS WITH TOP LAUNCHPADS, INFLUENCERS AND CRYPTO PROJECTS**
- **CONSTANTLY BUILDING TOOLS TO HELP INVESTORS DO BETTER RESEARCH**

Audits | KYCs | dApps
Contract Development

To hire us, reach out to
contact@spywolf.co or
t.me/joe_SpyWolf

FIND US ONLINE

SPYWOLF.CO

@SPYWOLFNETWORK

@SPYWOLFNETWORK

Disclaimer

This report shows findings based on our limited project analysis, following good industry practice from the date of this report, in relation to cybersecurity vulnerabilities and issues in the framework and algorithms based on smart contracts, overall social media and website presence and team transparency details of which are set out in this report. In order to get a full view of our analysis, it is crucial for you to read the full report.
While we have done our best in conducting our analysis and producing this report, it is important to note that you should not rely on this report and cannot claim against us on the basis of what it says or doesn't say, or how we produced it, and it is important for you to conduct your own independent investigations before making any decisions. We go into more detail on this in the disclaimer below – please make sure to read it in full.

DISCLAIMER:
By reading this report or any part of it, you agree to the terms of this disclaimer. If you do not agree to the terms, then please immediately cease reading this report, and delete and destroy any and all copies of this report downloaded and/or printed by you. This report is provided for information purposes only and on a non-reliance basis, and does not constitute investment advice.
No one shall have any right to rely on the report or its contents, and SpyWolf and its affiliates (including holding companies, shareholders, subsidiaries, employees, directors, officers and other representatives) (SpyWolf) owe no duty of care towards you or any other person, nor does SpyWolf make any warranty or representation to any person on the accuracy or completeness of the report.
The report is provided "as is", without any conditions, warranties or other terms of any kind except as set out in this disclaimer, and SpyWolf hereby excludes all representations, warranties, conditions and other terms (including, without limitation, the warranties implied by law of satisfactory quality, fitness for purpose and the use of reasonable care and skill) which, but for this clause, might have effect in relation to the report. Except and only to the extent that it is prohibited by law, SpyWolf hereby excludes all liability and responsibility, and neither you nor any other person shall have any claim against SpyWolf, for any amount or kind of loss or damage that may result to you or any other person (including without limitation, any direct, indirect, special, punitive, consequential or pure economic loss or damages, or any loss of income, profits, goodwill, data, contracts, use of money, or business interruption, and whether in delict, tort (including without limitation negligence), contract, breach of statutory duty, misrepresentation (whether innocent or negligent) or otherwise under any claim of any nature whatsoever in any jurisdiction) in any way arising from or connected with this report and the use, inability to use or the results of use of this report, and any reliance on this report. The analysis of the security is purely based on the smart contracts, website, social media and team.
No applications were reviewed for security. No product code has been reviewed.